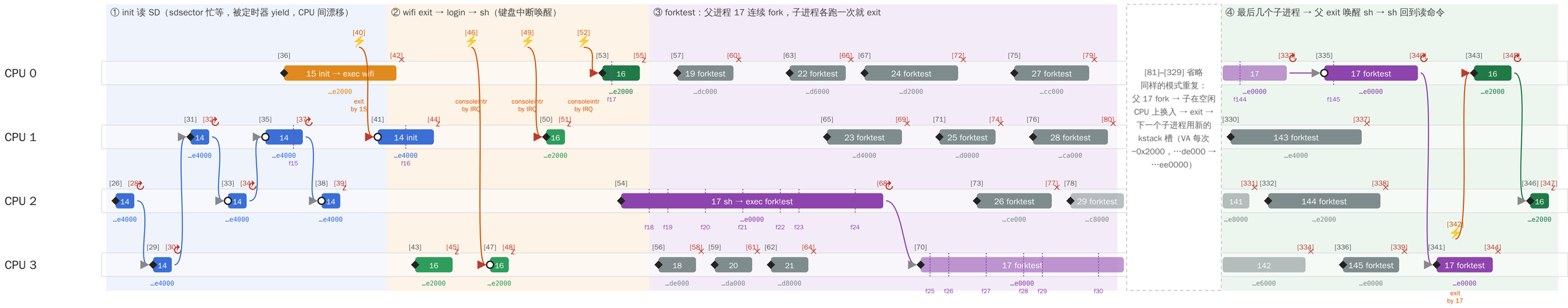
图 AO　log8（仅用户进程）：init → wifi → login → sh → forktest（128 个子进程）的调度、栈和换出时的 context 对比

横轴 = 全局编号（内核线程记录已隐藏，[81]–[329] 中间 ~250 条 forktest 重复记录折叠）；色条 = 该 CPU 上两条记录之间运行的任务，条下小字 = 这段时间 sp 所在的内核栈（kstack VA 低位）



◆ 换入（新进程或换了 CPU）　○ again-in（同一 CPU 再选中同一任务）　红色 [n] = 换出：↻ 被抢占（slice 用完，从定时器 IRQ yield）　z 睡眠　× exit　⚡ 唤醒（橙箭头指向被唤醒任务的下一次换入）　灰箭头 = 被抢占后在哪个 CPU 继续　紫虚线 fN = 父进程 fork 出 pid N

颜色：蓝 init(14)　橙 wifi(15)　绿 login/sh(16)　紫 forktest 父(17)　灰 forktest 子(18…145)。注意 pid 15 和 16 用同一个 kstack VA …e2000（wifi 退出后槽位复用），每个 forktest 子进程则各占一个新槽。

8 次切换的栈与 context：每次换到不同进程时，swtch 两侧的栈长什么样、p->context 里每个寄存器指向哪里（全部来自 log8 的实时记录；红线 = context.sp / x29 指向栈上的位置）

① [28] init 被抢占（CPU2 换出）

kstack VA 0x…ffe4000–e5000 PA 06f9d000 proc 0x…fa4900

sdsector 忙等时定时器 IRQ 打在 init 自己的内核栈上，el1irq 存现场后 yield → sched → swtch。



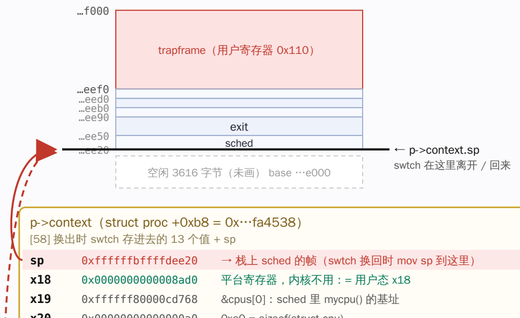
p->context（struct proc +0xb8 = 0x…fa49b8）
[28] 换出时 swtch 存进去的 13 个值 + sp

sp	0xffffffbffffe49d0	→ 栈上 sched 的帧（swtch 换回时 mov sp 到这里）
x18	0x0000000000000000	0
x19	0xffffff80000cd768	&cpus[0]：sched 里 mycpu() 的基址
x20	0x00000000000000a0	0xa0 = sizeof(struct cpu)
x21	0xffffff8006fa4900	p 自己（pid 14 的 struct proc）
x22	0x0000000000000000	intena=0：从定时器 IRQ 里 yield（中断已关）
x23	0x0000000000000001	调用链局部变量（= 1）
x24	0xffffff80000f29c8	&sd_rca：sdsector 用的 SD 卡相对地址变量
x25	0xffffff803f20203c	SD host MMIO 0x3f20203c：正在轮询的寄存器
x26	0x0000000000000000	调用链局部变量（= 0）
x27	0xffffff803f202050	SD host MMIO 0x3f202050：正在轮询的寄存器
x28	0x0000000000000001	调用链局部变量（= 1）
x29 fp	0xffffffbffffe49d0	= sp：sched 帧记录 {调用者 fp, lr}，回溯起点
x30 lr	0xffffff8000083ac0	sched+0xb4：bl swtch 的下一条，换入后 ret 到这里

② [42] wifi exit（CPU0 换出）

kstack VA 0x…ffe2000–e3000 PA 06ee5000 proc 0x…fa4780

exit → sched，永不再换入。栈很浅：只有系统调用链。



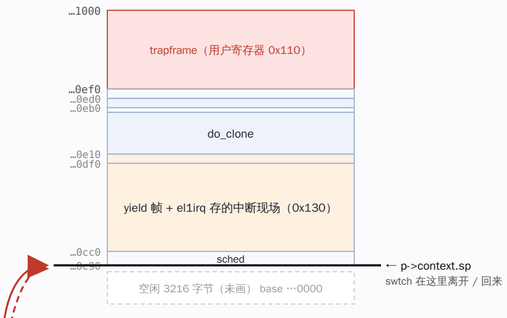
p->context（struct proc +0xb8 = 0x…fa4838）
[42] 换出时 swtch 存进去的 13 个值 + sp

sp	0xffffffbffffe2e20	→ 栈上 sched 的帧（swtch 换回时 mov sp 到这里）
x18	0x0000000000008ad0	平台寄存器，内核不用：= 用户态 x18
x19	0xffffff80000cd768	&cpus[0]：sched 里 mycpu() 的基址
x20	0x00000000000000a0	0xa0 = sizeof(struct cpu)
x21	0xffffff8006fa4780	p 自己（pid 15 的 struct proc）
x22	0x0000000000000001	intena=1：从 sleep/exit 系统调用路径进来
x23	0x0000000000000000	调用链局部变量（= 0）
x24	0xffffff80000cdad8	&wait_lock：exit/wait 用的锁
x25	0x0000000000000440	= trapframe.x25：用户态的值，这条路径没用到
x26	0x0000000000008ad0	= trapframe.x26：用户态的值，这条路径没用到
x27	0x0000003fefffbb8	= trapframe.x27：用户态的值，这条路径没用到
x28	0x0000000000008b58	= trapframe.x28：用户态的值，这条路径没用到
x29 fp	0xffffffbffffe2e20	= sp：sched 帧记录 {调用者 fp, lr}，回溯起点
x30 lr	0xffffff8000083ac0	sched+0xb4：bl swtch 的下一条，换入后 ret 到这里

③ [45] login 睡眠（CPU3 换出）

kstack VA 0x…ffe2000–e3000 PA 06ee5000 proc 0x…fa4780

read(0) → consoleread → sleep(&cons.r)。[46] 键盘 IRQ 唤醒，[47] 同 CPU again-in。



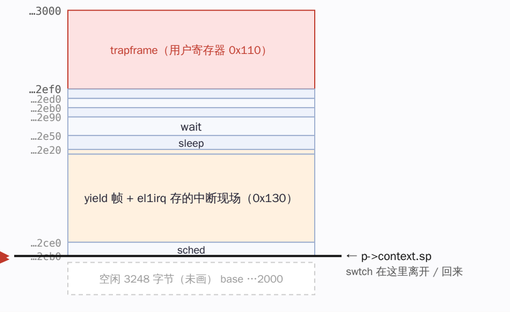
p->context（struct proc +0xb8 = 0x…fa4838）
[45] 换出时 swtch 存进去的 13 个值 + sp

sp	0xffffffbffffe2dc0	→ 栈上 sched 的帧（swtch 换回时 mov sp 到这里）
x18	0x0000000000008ad0	平台寄存器，内核不用：= 用户态 x18
x19	0xffffff80000cd768	&cpus[0]：sched 里 mycpu() 的基址
x20	0x00000000000000a0	0xa0 = sizeof(struct cpu)
x21	0xffffff8006fa4780	p 自己（pid 16 的 struct proc）
x22	0x0000000000000001	intena=1：从 sleep/exit 系统调用路径进来
x23	0x0000000000000020	consoleread 的 n = 32（read 请求字节数）
x24	0x0000000000000001	user_dst = 1（拷到用户地址）
x25	0xffffff8000117bc8	&cons+0x98：= sleep 的 chan（cons.r）
x26	0x0000000000008ad0	= trapframe.x26：用户态的值，这条路径没用到
x27	0x0000003fefffbb8	= trapframe.x27：用户态的值，这条路径没用到
x28	0x0000000000008b58	= trapframe.x28：用户态的值，这条路径没用到
x29 fp	0xffffffbffffe2dc0	= sp：sched 帧记录 {调用者 fp, lr}，回溯起点
x30 lr	0xffffff8000083ac0	sched+0xb4：bl swtch 的下一条，换入后 ret 到这里

④ [55] sh 在 wait 里睡（CPU0 换出）

kstack VA 0x…ffe2000–e3000 PA 06ee5000 proc 0x…fa4780

sh fork 了 17（forktest），wait → sleep(chan = sh 自己的 proc)。[342] 由 17 的 exit 唤醒。



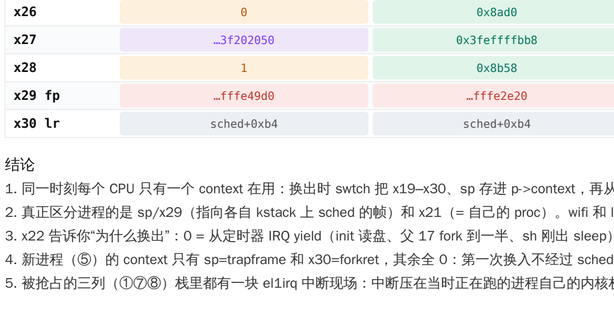
p->context（struct proc +0xb8 = 0x…fa4838）
[55] 换出时 swtch 存进去的 13 个值 + sp

sp	0xffffffbffffe2df0	→ 栈上 sched 的帧（swtch 换回时 mov sp 到这里）
x18	0x0000000000009ac0	平台寄存器，内核不用：= 用户态 x18
x19	0xffffff80000cd768	&cpus[0]：sched 里 mycpu() 的基址
x20	0x00000000000000a0	0xa0 = sizeof(struct cpu)
x21	0xffffff8006fa4780	p 自己（pid 16 的 struct proc）
x22	0x0000000000000001	intena=1：从 sleep/exit 系统调用路径进来
x23	0x0000000000000000	调用链局部变量（= 0）
x24	0x000000000000001e	= trapframe.x24：用户态的值，这条路径没用到
x25	0x00000000000008d0	= trapframe.x25：用户态的值，这条路径没用到
x26	0x00000000000009ac0	= trapframe.x26：用户态的值，这条路径没用到
x27	0x0000003fefffbb8	= trapframe.x27：用户态的值，这条路径没用到
x28	0x00000000000009b48	= trapframe.x28：用户态的值，这条路径没用到
x29 fp	0xffffffbffffe2df0	= sp：sched 帧记录 {调用者 fp, lr}，回溯起点
x30 lr	0xffffff8000083ac0	sched+0xb4：bl swtch 的下一条，换入后 ret 到这里

⑤ [56] forktest 子 18 首次换入（CPU3）

kstack VA 0x…ffde000–df000 PA 06ec4000 proc 0x…fa4480

新 context：fork 时只设 sp 和 x30，其余全 0。swtch 装入后 ret 到 forkret → 回用户态。



p->context（struct proc +0xb8 = 0x…fa4538）
[56] 换入前，swtch 要装入的 13 个值 + sp

sp	0xffffffbffffdeef0	= trapframe 地址：首次换入后栈从这里开始
x18	0x0000000000000000	0（新 context 不需要）
x19	0x0000000000000000	0（新 context 不需要）
x20	0x0000000000000000	0（新 context 不需要）
x21	0x0000000000000000	0（新 context 不需要）
x22	0x0000000000000000	0（新 context 不需要）
x23	0x0000000000000000	0（新 context 不需要）
x24	0x0000000000000000	0（新 context 不需要）
x25	0x0000000000000000	0（新 context 不需要）
x26	0x0000000000000000	0（新 context 不需要）
x27	0x0000000000000000	0（新 context 不需要）
x28	0x0000000000000000	0（新 context 不需要）
x29 fp	0x0000000000000000	0：帧链从空开始
x30 lr	0xffffff8000083078	forkret：ret 到 forkret → usertrapret → eret

⑥ [58] 子 18 exit（CPU3 换出）

kstack VA 0x…ffde000–df000 PA 06ec4000 proc 0x…fa4480

fork 返回 0 → 立刻 exit → sched。和②wifi 的 exit 栈形状完全一样，只是换了一块栈。



p->context（struct proc +0xb8 = 0x…fa4538）
[58] 换出时 swtch 存进去的 13 个值 + sp

sp	0xffffffbffffdee20	→ 栈上 sched 的帧（swtch 换回时 mov sp 到这里）
x18	0x0000000000008ad0	平台寄存器，内核不用：= 用户态 x18
x19	0xffffff80000cd768	&cpus[0]：sched 里 mycpu() 的基址
x20	0x00000000000000a0	0xa0 = sizeof(struct cpu)
x21	0xffffff8006fa4480	p 自己（pid 18 的 struct proc）
x22	0x0000000000000001	intena=1：从 sleep/exit 系统调用路径进来
x23	0x0000000000000000	调用链局部变量（= 0）
x24	0xffffff80000cdad8	&wait_lock：exit/wait 用的锁
x25	0x0000000000000358	= trapframe.x25：用户态的值，这条路径没用到
x26	0x0000000000008ad0	= trapframe.x26：用户态的值，这条路径没用到
x27	0x0000003fefffbb8	= trapframe.x27：用户态的值，这条路径没用到
x28	0x0000000000008b58	= trapframe.x28：用户态的值，这条路径没用到
x29 fp	0xffffffbffffdee20	= sp：sched 帧记录 {调用者 fp, lr}，回溯起点
x30 lr	0xffffff8000083ac0	sched+0xb4：bl swtch 的下一条，换入后 ret 到这里

⑦ [68] forktest 父 17 在 fork 中被抢占（CPU2）

kstack VA 0x…ffe0000–e1000 PA 06ed0000 proc 0x…fa4600

do_clone 复制到一半来了定时器 IRQ，yield 换出；[70] 在 CPU3 换入接着复制。



p->context（struct proc +0xb8 = 0x…fa46b8）
[68] 换出时 swtch 存进去的 13 个值 + sp

sp	0xffffffbffffe0c90	→ 栈上 sched 的帧（swtch 换回时 mov sp 到这里）
x18	0x0000000000008ad0	平台寄存器，内核不用：= 用户态 x18
x19	0xffffff80000cd768	&cpus[0]：sched 里 mycpu() 的基址
x20	0x00000000000000a0	0xa0 = sizeof(struct cpu)
x21	0xffffff8006fa4600	p 自己（pid 17 的 struct proc）
x22	0x0000000000000000	intena=0：从定时器 IRQ 里 yield（中断已关）
x23	0x0000000000000000	调用链局部变量（= 0）
x24	0xffffff8006ff9460	files 结构：do_clone 正在给新子进程（pid 25）复制
x25	0xffffff8006ff82c0	fs 结构：do_clone 正在给新子进程（pid 25）复制
x26	0x0000000000000000	调用链局部变量（= 0）
x27	0xffffff8006ffb5e8	pid 17 的 vm 结构 +0x48
x28	0x0000000000000000	调用链局部变量（= 0）
x29 fp	0xffffffbffffe0c90	= sp：sched 帧记录 {调用者 fp, lr}，回溯起点
x30 lr	0xffffff8000083ac0	sched+0xb4：bl swtch 的下一条，换入后 ret 到这里

⑧ [345] sh 在 sleep 里被抢占（CPU0）

kstack VA 0x…ffe2000–e3000 PA 06ee5000 proc 0x…fa4780

[342] 被唤醒、[343] 换入后，刚从 sleep 出来还没返回就被 IRQ yield；[346] 换到 CPU2。



p->context（struct proc +0xb8 = 0x…fa4838）
[345] 换出时 swtch 存进去的 13 个值 + sp

sp	0xffffffbffffe2cb0	→ 栈上 sched 的帧（swtch 换回时 mov sp 到这里）
x18	0x0000000000009ac0	平台寄存器，内核不用：= 用户态 x18
x19	0xffffff80000cd768	&cpus[0]：sched 里 mycpu() 的基址
x20	0x00000000000000a0	0xa0 = sizeof(struct cpu)
x21	0xffffff8006fa4780	p 自己（pid 16 的 struct proc）
x22	0x0000000000000000	intena=0：从定时器 IRQ 里 yield（中断已关）
x23	0x0000000000000000	调用链局部变量（= 0）
x24	0x000000000000001e	= trapframe.x24：用户态的值，这条路径没用到
x25	0x00000000000008d0	= trapframe.x25：用户态的值，这条路径没用到
x26	0x00000000000009ac0	= trapframe.x26：用户态的值，这条路径没用到
x27	0x0000003fefffbb8	= trapframe.x27：用户态的值，这条路径没用到
x28	0x00000000000009b48	= trapframe.x28：用户态的值，这条路径没用到
x29 fp	0xffffffbffffe2cb0	= sp：sched 帧记录 {调用者 fp, lr}，回溯起点
x30 lr	0xffffff8000083ac0	sched+0xb4：bl swtch 的下一条，换入后 ret 到这里

context 横向对比：同一个寄存器在不同进程/不同换出原因下的值

颜色 = 含义类别：灰 = 所有 live context 都一样　红 = 指向自己的内核栈　紫 = 自己的 proc / 内核数据　橙 = 由换出路径决定　绿 = 原样带着的用户态值　浅灰 = 新 context 里的 0

寄存器	[28] 14 init 被抢占 · el1irq	[42] 15 wifi exit · sys_exit	[45] 16 login sleep · consoleread	[55] 16 sh sleep · wait	[56] 18 forktest 首次换入 · 新 context	[58] 18 forktest exit · sys_exit	[68] 17 forktest 被抢占 · el1irq	[345] 16 sh 被抢占 · el1irq	规律
sp	…fffe49d0	…fffe2e20	…fffe2dc0	…fffe2df0	…fffdeef0	…fffdee20	…fffe0c90	…fffe2cb0	每个进程指向自己 kstack 里 sched 的帧；深浅取决于调用链
x18	0	0x8ad0	0x8ad0	0x9ac0	0	0x8ad0	0x8ad0	0x9ac0	内核不用：live 时带着用户态 x18，新 context 为 0
x19	cpus	cpus	cpus	cpus	0	cpus	cpus	cpus	live 一律 &cpus[0]（sched 用来算 mycpu）
x20	0xa0	0xa0	0xa0	0xa0	0	0xa0	0xa0	0xa0	live 一律 0xa0 = sizeof(struct cpu)
x21	…06fa4900	…06fa4780	…06fa4780	…06fa4780	0	…06fa4480	…06fa4600	…06fa4780	live 一律 = p 自己：每列不同，等于各自 proc 地址
x22	0	1	1	1	0	1	0	0	intena：IRQ 抢占进来 = 0，sleep/exit 进来 = 1
x23	1	0	0x20	0	0	0	0	0	调用链里的局部变量（consoleread 的 n 等）
x24	sd_rca	wait_lock	1	0x1e	0	wait_lock	…06ff9460	0x1e	路径决定：&sd_rca / &wait_lock / user_dst / 新子进程的 files…
x25	…3f20203c	0x440	cons+0x98	0x8d0	0	0x358	…06ff82c0	0x8d0	路径决定：SD MMIO / &cons.r / fs 结构；没用到时是用户值
x26	0	0x8ad0	0x8ad0	0x9ac0	0	0x8ad0	0	0x9ac0	大多是用户态 x26 原样带下来（callee-saved 没人改）
x27	…3f202050	0x3fefffbb8	0x3fefffbb8	0x3fefffbb8	0	0x3fefffbb8	…06ffb5e8	0x3fefffbb8	同上；init 读盘时是 SD MMIO，fork 时是父 vm 结构
x28	1	0x8b58	0x8b58	0x9b48	0	0x8b58	0	0x9b48	同上
x29 fp	…fffe49d0	…fffe2e20	…fffe2dc0	…fffe2df0	0	…fffdee20	…fffe0c90	…fffe2cb0	= sp：sched 帧记录的地址，回溯起点
x30 lr	sched+0xb4	sched+0xb4	sched+0xb4	sched+0xb4	forkret	sched+0xb4	sched+0xb4	sched+0xb4	live 一律 sched+0xb4；新 context = forkret

结论

1. 同一时刻每个 CPU 只有一个 context 在用：换出时 swtch 把 x19–x30、sp 存进 p->context，再从 c->context 装回调度器；换入反过来。所以 8 列里"灰色"那几行（x19/x20/x30）永远一样 —— 它们是 sched() 自己的寄存器，不属于进程。
2. 真正区分进程的是 sp/x29（指向各自 kstack 上 sched 的帧）和 x21（= 自己的 proc）。wifi 和 login/sh 的 sp 都在 …e2xxx，是同一个 VA 槽先后给了 pid 15 和 16；forktest 的子进程每个占一个新槽（…de000、…dc000 …）。
3. x22 告诉你"为什么换出"：0 = 从定时器 IRQ yield（init 读盘、父 17 fork 到一半、sh 刚出 sleep），1 = 自己走 sleep/exit。x23–x28 是换出路径上的 callee-saved 寄存器，路径没用到的就原样保留用户态的值（绿色）。
4. 新进程（⑤）的 context 只有 sp=trapframe 和 x30=forkret，其余全 0：第一次换入不经过 sched，直接 ret 到 forkret 回用户态。它 exit（⑥）时的栈形状与 wifi exit（②）完全相同。
5. 被抢占的三列（①⑦⑧）栈里都有一块 el1irq 中断现场：中断压在当时正在跑的进程自己的内核栈上，下次在别的 CPU 换入（[29]/[70]/[346]）时从这里 eret 回去，所以"换 CPU 继续"不需要搬栈。